



Alleviating the quantum Big- M problem



Edoardo Alessandrini^{1,2}✉, Sergi Ramos-Calderer^{1,3}, Ingo Roth¹, Emiliano Traversi⁴ & Leandro Aolita¹

A major obstacle for quantum optimizers is the reformulation of constraints as a quadratic unconstrained binary optimization (QUBO). Current QUBO translators exaggerate the weight M of the penalty terms. Classically known as the “Big- M ” problem, the issue becomes even more daunting for quantum solvers, since it affects the physical energy scale. We take a systematic, encompassing look at the quantum big- M problem, revealing NP-hardness in finding the optimal M and establishing bounds on the Hamiltonian spectral gap Δ as a function of the weight M , inversely related to the expected run-time of quantum solvers. We propose a practical translation algorithm, based on SDP relaxation, that outperforms previous methods in numerical benchmarks. Our algorithm gives values of Δ orders of magnitude greater, e.g. for portfolio optimization instances. Solving such instances with an adiabatic algorithm on 6-qubits of an IonQ device, we observe significant advantages in time to solution and average solution quality. Our findings are relevant to quantum and quantum-inspired solvers alike.

Quantum computing holds a great potential for speeding up combinatorial optimization¹. From a distant-future perspective, the prospects are rooted in the fact that fault-tolerant quantum computers are envisioned to run quantum versions of state-of-the-art classical optimization algorithms more efficiently. In fact, there is sound theoretical evidence that such quantum algorithms offer a quadratic asymptotic speed-up over their classical counterparts^{2–4}. In the short run, there is a direct relation between the ground state of physical systems and optimizations. The paradigmatic example is Ising models encoding quadratic *unconstrained* binary optimization (QUBO) problems. This has fueled a quest for ground-state preparation algorithms implementable on nearer-term quantum hardware. These include quantum annealing^{5–10}, quantum imaginary time evolution^{11–18}, and heuristics such as the quantum approximate optimization algorithm^{19–21}. Moreover, apart from quantum solvers, the QUBO paradigm is giving rise to a variety of interesting quantum-inspired solvers as well^{22–24}.

A prerequisite to apply such paradigm to more general quadratically constrained integer optimization problems is to recast them into an equivalent QUBO form. Recently, automatic QUBO translators have appeared^{25–27}. The translation consists of lifting the constraints to penalty terms in the objective function. To ensure that the solution to the reformulated (unconstrained) problem coincides with that of the original (constrained) one, the weight of the penalty terms—often denoted as M —has to be sufficiently large. At the same time, choosing an excessively large M causes an increase in run-time due to precision issues in rounding and truncation, even for classical solvers. In the classical optimization community this problem is referred to as the *Big- M problem*.

In contrast, quantum QUBO solvers are closer in spirit to *analog computing devices*. There, the value of M directly affects the energy scale of the Hamiltonian whose ground state encodes the solution. When performing controlled computations with an actual physical quantum system its admissible energies are limited. For ground-state preparation schemes these physical limitation eventually also restricts the energy scale of the encoding Hamiltonian^{28–30}. More precisely, the penalty terms tend to have the undesired side effect of decreasing the spectral gap of the Hamiltonian. As a consequence, the precision required to resolve the states, and, hence, also the run-time, increases. A general rule of thumb is to choose M as small as possible, but doing so while still successfully enforcing the constraints is highly non-trivial. In fact, the known computationally-efficient Big- M recipes tend to largely over-estimate the required value^{25,29,31}. Clearly, an efficient QUBO translator with improved spectral properties is highly desirable. Moreover, a formalization of the *quantum big- M problem* as a fundamental concept between quantum physics and computer science is missing too. A general framework should address key aspects such as how to quantify the Big- M problem in terms of its impact on quantum solvers or the computational complexity of QUBO reformulations.

Here, we fill in this gap. We develop a theory of the quantum big- M problem and its impact on the spectral gap Δ . We start with rigorous definitions for the notions of optimal M and exact QUBO reformulations. We prove that finding the optimal M is NP-hard and establish relevant upper bounds on the Hamiltonian spectral gap Δ , both in terms of the original gap and of M . One of these bounds formalizes the intuition that $\Delta = \mathcal{O}(M^{-1})$. Most importantly, we present a universal QUBO reformulation method with improved spectral properties. This is a simple but

¹Quantum Research Centre, Technology Innovation Institute (TII), Abu Dhabi, UAE. ²SISSA—Scuola Internazionale Superiore di Studi Avanzati, Trieste, Italy.

³Departament de Física Quàntica i Astrofísica and Institut de Ciències del Cosmos (ICCUB), Universitat de Barcelona, Barcelona, Spain. ⁴Department of Information Systems, Data Analytics and Operations, ESSEC Business School, Cergy-Pontoise, France. ✉e-mail: ealessan@sisssa.it

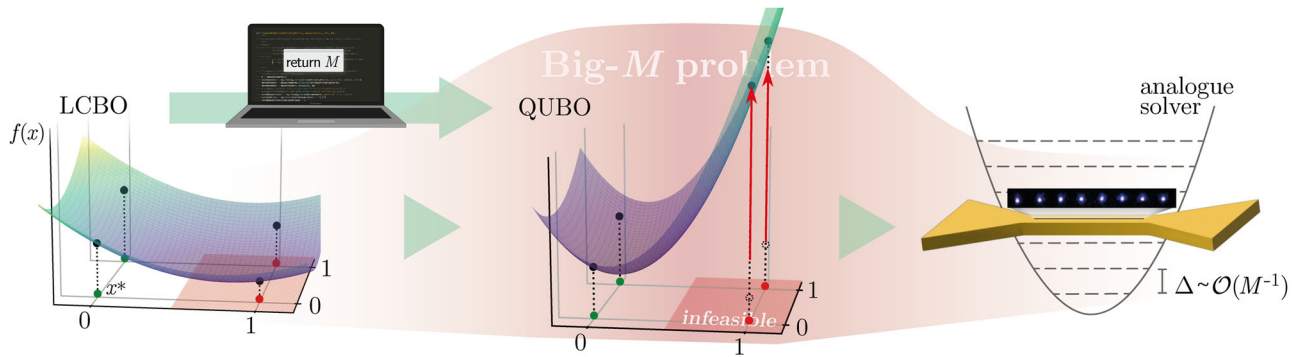


Fig. 1 | In a linearly-constrained binary quadratic optimization (LCBO, left) with optimal point $\mathbf{x}^*$, unfeasible points (red-shaded sub-domain) are excluded from the feasible set (green) via hard constraints. For a reformulation as a quadratic unconstrained binary optimization (QUBO, center), the native format for quantum solvers, a penalty term that vanishes on the feasible region with weight $M > 0$ is added to the objective function. The reformulation is exact ($\mathbf{x}^*$ remains optimal) if M is

sufficiently large, lifting the objective values of all unfeasible points above $f(\mathbf{x}^*)$. However, the penalty term affects the physical energy scale; the larger M , the smaller the spectral gap Δ of the Hamiltonian encoding the reformulation (right). This has a detrimental effect on the runtime of exact solvers, as well as on the quality of the solution of approximate solvers. While NP-hard in general (as we prove), we present an efficient classical strategy to find ‘good’ choices of M .

remarkably-powerful heuristic recipe for M , based on standard SDP relaxation³². We perform exhaustive numerical tests on sparse linearly constrained binary optimizations, set partition problems, and portfolio optimizations from real S&P 500 data. For all three classes, we systematically obtain values of M one order of magnitude smaller and of Δ from one to two orders of magnitude larger than with state-of-the-art methods, with particularly promising results for portfolio optimization. In addition, in a proof-of-principle experiment, we solved 6-qubit PO instances with a Trotterized adiabatic algorithm deployed on IonQ’s trapped-ion device Aria-1. For the small system size one remains approximately adiabatic with the permissible circuit depth. We observed that our reformulation increases the probability of measuring the optimal solution by over an order of magnitude and improves the average approximation ratio. Our findings demonstrate crucial advantages of the proposed optimized QUBO reformulation over the currently known recipes.

Results

The quantum Big- M problem

Our starting point is a *linearly-constrained binary quadratic optimization (LCBO) problem* with n binary decision variables and m constraints,

$$\underset{\mathbf{x} \in \{0,1\}^n}{\text{minimize}} \quad f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x} \quad \text{subject to} \quad \mathbf{A} \mathbf{x} = \mathbf{b}, \quad (\text{P})$$

specified in terms of $\mathbf{Q} \in \mathbb{Z}^{n \times n}$, $\mathbf{A} \in \mathbb{Z}^{m \times n}$, and $\mathbf{b} \in \mathbb{Z}^m$. Note that a general polynomially-constrained polynomial optimization problem with integer variables can always be cast into a linearly-constrained binary quadratic optimization problem of the form (P) by standard *gadgets*, as summarized in Supplementary Information A. Furthermore, for the sake of clarity, we consider throughout *exact optimization solvers*. Clearly, near-term quantum optimization solvers are envisioned to be approximate solvers. However, our discussion can be extended to approximate solvers too, e.g. by considering all admissible approximate solutions as optimal points of problem (P).

To arrive at a QUBO formulation of (P), the best-known strategy (see Fig. 1) is to promote the constraints to a quadratic penalty term in the objective function using a suitable constant weight $M > 0$. The resulting QUBO reads,

$$\underset{\mathbf{x} \in \{0,1\}^n}{\text{minimize}} \quad \mathbf{x}^T \mathbf{Q} \mathbf{x} + M(\mathbf{A} \mathbf{x} - \mathbf{b})^2. \quad (\text{PM})$$

We say that (P_M) is an *exact reformulation* of (P) if their optimal points coincide. The penalty term in (P_M) vanishes for every feasible point. To

arrive at an exact reformulation, M has to be chosen large enough for every unfeasible point of (P) to have a greater objective value than the original optimum. Denoting by $\mathbf{x}^*$ an optimal point of (P), we have an exact reformulation if and only if there exists a $\text{gap} \delta > 0$ s.t.

$$f(\mathbf{x}^*) + \delta \leq f(\mathbf{x}) + M(\mathbf{A} \mathbf{x} - \mathbf{b})^2 \quad (1)$$

for all unfeasible points $\mathbf{x}$. There are simple choices of M to ensure this condition, such as

$$M_{\ell_1} = \|\mathbf{Q}\|_{\ell_1} + \delta, \quad (2)$$

with the vector ℓ_1 -norm being the sum of all absolute entries. Since M_{ℓ_1} can be computed in polynomial time, it follows that (P) and (P_M) with $M = M_{\ell_1}$ are in the same complexity class. This choice of M is common^{25,29,31}; but, as we show below, it typically yields excessively large values.

We say that a reformulation (P_M) of (P) has a (δ) -optimal M if it is exact with gap δ and minimal M . Note that the minimal choice of M guarantees only a difference δ between the optimal objective value and those of the unfeasible points. To avoid an arbitrarily small gap, δ can be chosen as a constant independent on the system size in Eq. (1). For specific classes of problems it is in fact possible to formulate strategies for optimal choices of M , an example being the problem of finding maximum independent sets, where the optimal value of M is apparent³³. In general, however, this is intractable.

Observation 1. Finding an optimal M is NP-hard. Intuitively, Eq. (1) already hints at the possibility that finding the optimal M can be as hard as determining the optimal objective value of the original optimization problem. In Section “On the hardness of the quantum Big- M problem”, we give a polynomial reduction of the problem of deciding if the optimum of f is below a threshold to the problem of deciding if a given M provides an exact reformulation.

From a pragmatic point of view though, it is nonetheless of utmost importance to find suboptimal but ‘good’ choices of M using less resources than required for solving the original problem. In some specific cases (Travelling Salesman Problem³⁴, permutation problems³⁵), there are recipes for a ‘reasonable’ value for M . Here, we provide a generally applicable strategy to determine ‘good’ choices of M . At the heart of our approach is the following observation.

Observation 2. Let $\mathbf{x}_{\text{feas}}$ be a feasible point of (P), $\delta > 0$, and choose $f_{\text{unc}} \leq \min\{f(\mathbf{x}) \mid \mathbf{x} \in \{0,1\}^n\}$, i.e. as a lower bound on the objective

function of (P) when omitting the linear constraints. Then, (P_M) with

$$M = f(\mathbf{x}_{\text{feas}}) - f_{\text{unc}} + \delta \quad (3)$$

is an exact reformulation of (P) with gap (at least) δ .

Proof. Let $\mathbf{x}$ be an unfeasible point and M chosen according to Eq. (3). Since $(A\mathbf{x} - \mathbf{b})^2 \geq 1$, $M(A\mathbf{x} - \mathbf{b})^2 \geq f(\mathbf{x}_{\text{feas}}) - f_{\text{unc}} + \delta \geq f(\mathbf{x}^*) - f(\mathbf{x}) + \delta$. The second inequality follows from the definition of f_{unc} and the fact that $f(\mathbf{x}^*) \leq f(\mathbf{x}_{\text{feas}})$ for any feasible point $\mathbf{x}_{\text{feas}}$. Thus, Eq. (1) holds.

While any choice of feasible point and lower bound yields an admissible value of M , good choices of M attempt to choose the $\mathbf{x}_{\text{feas}}$ with small objective $f(\mathbf{x}_{\text{feas}})$ and the bound f_{unc} as tight as possible. A universal strategy to this end is the following: i) Find a feasible point $\mathbf{x}_{\text{feas}}$ by running a classical solver on (P) limited to some constant amount of time. ii) Solve the Semidefinite Programming (SDP) relaxation (see Supplementary Information B) of the unconstrained minimization of f . Use the resulting objective value as f_{unc} . This strategy is our main numerical tool. We denote the value given by it as M_{SDP} .

We note that there exist problem instances where even finding a feasible point is hard. In practice, however, there exist efficient heuristics to determine feasible points. The underlying mindset of our strategy is that modern classical solvers can be powerful allies to quantum optimizers, e.g. performing tractable pre-computations to optimize the reformulation for the quantum hardware. As for concluding potential advantages of quantum solvers over classical solvers, with this strategy, one must of course be particularly careful not to accidentally reduce the complexity of the problem in the pre-computation. In the case of exact reformulations with optimized M , we expect the complexity not to decrease even if an optimal M is provided. This expectation is supported by the following analysis.

Spectral gap as a measure of the Big- M problem

Near-term quantum (and quantum-inspired) solvers are based on ground-state optimization of an Ising Hamiltonian $H_M = H_f + M H_c$ (see Section “Experimental implementation” for explicit expressions) that encodes the objective function in (P_M) . The Hamiltonian H_f encodes the objective function $f(\mathbf{x})$ of the original problem, while H_c encodes the constraint term $(A\mathbf{x} - \mathbf{b})^2$. Hence, the choice of M directly affects the spectral gap

$$\Delta_M := \frac{E_1 - E_0}{E_{\text{max}} - E_0}, \quad (4)$$

where E_0 , E_1 , and E_{max} are respectively the lowest, next-to-lowest, and maximum energies of H_M . The spectral gap normalization is imperative to realistically compare different Hamiltonian. A physical quantum solver has access to a restricted energy scale that the problem Hamiltonian we want to solve must accommodate to. From a dual perspective, this energy scale relates to numerical precision, as it evaluates the ratio between the accuracy needed to discriminate the lowest-energy state, and the full energy spectrum.

The connection between the spectral gap and M is made apparent through the following calculations (Section “Bounds on the spectral gap of Big- M QUBO reformulations”).

Observation 3. If (P_M) is an exact reformulation, then i) $E_0 = f(\mathbf{x}^*)$; ii) $\Delta_M \leq \Delta_0$ for all M , with Δ_0 the ‘spectral gap’ of the constrained optimization problem (P); and iii) $\Delta_M/\Delta_0 \leq (E_{\text{max}}^{(f)} - E_0)/(E_{\text{max}}^{(c)} M - M^*)$ where M^* is an optimal M , as defined in the previous section, and $E_{\text{max}}^{(c)}$ and $E_{\text{max}}^{(f)}$ are the maximum energies of H_c and H_f respectively. The first implication states simply that E_0 equals the optimal objective value of problem (P). The second one that no exact reformulation (P_M) can increase the spectral gap. Finally, since both H_f and H_c are independent of M , the bound in iii) implies that $\Delta_M \in \mathcal{O}(\Delta_0/M)$ asymptotically. This substantiates the initial intuition that an excessively high penalty M is detrimental to analogue solvers. For instance, in quantum annealers, adiabaticity requires a run-time $\Omega(\Delta_M^{-2})$ ^{5,6}. In turn, for imaginary time evolution, the inverse temperature required for

constant-error ground-state approximation is $\Omega(\Delta_M^{-1})$ ^{11–18}. For variational algorithms, also the training is affected: as M grows, the sensitivity of the cost function (the energy) to parameter changes becomes increasingly dominated by H_c and H_f increasingly irrelevant^{29,31}. Hence, we propose Δ_M as a natural measure for the Big- M problem of a QUBO reformulation. This allows us to quantitatively benchmark our Big- M recipe against the previous direct bounds, which we do next.

Numerical benchmarks

We evaluate the performance of reformulations with optimized M_{SDP} , against the common choice M_{ℓ_1} for three examples of LCBO problem classes: Random sparse LCBOs, set partitioning problems (SPPs), and portfolio optimization (PO). Details on the model definitions and further results are presented in Section “Benchmarked models”.

For PO we use the well-known Markovitz model^{36–38}, i.e. the problem of selecting a set of assets maximizing returns while minimizing risk. The problem specification requires a vector $\boldsymbol{\mu}$ of expected returns of a set of N assets, their covariance matrix Σ , a risk aversion $\gamma > 0$, and a partition number w defining the portfolio discretization. Denoting by x_i the units of asset i in the portfolio, the problem formulation reads

$$\underset{\mathbf{x} \in \mathbb{N}^N}{\text{minimize}} \quad -\boldsymbol{\mu}'\mathbf{x} + \gamma \mathbf{x}^T \Sigma \mathbf{x} \quad \text{subject to} \quad \sum_i x_i = 2^w - 1. \quad (5)$$

The constraint forces the total budget to be invested. The QUBO reduction requires mapping each integer decision variable into w binary variables. We generate problem instances from historic financial data on S&P 500 stocks.

We observe that the result M_{SDP} of our algorithm is consistently one order of magnitude smaller than M_{ℓ_1} for random sparse LCBOs [Fig. 2a] and SPPs [Fig. 3 in Section “Benchmarked models”]. Concomitantly, the spectral gap is relatively increased by an order of magnitude [Fig. 2d]. This corroborates our theoretic consideration relating optimized choices of M to significantly improved spectral properties of the QUBO formulation. In the PO instances we additionally observe that the advantage in M and Δ further grows with the problem size [Fig. 2b, e]. In contrast to random LCBOs and SPPs, PO has a single constraint independent of the problem size. While this allows for highly optimized choices of the penalty weight as exemplified by M_{SDP} , the bound in Eq. (2) is oblivious to the intrinsic structure of PO. Using a greedy heuristic to determine $f(\mathbf{x}_{\text{feas}})$ (Section “Greedy algorithm for Portfolio Optimization”), we further calculate M_{SDP} for PO instances with up to 300 binary variables and find that the improvements over M_{ℓ_1} persist [Fig. 2c].

Quantum hardware deployment

Finally, after building our theoretical framework and numerical methods, we turn to the question of how relevant the Big- M problem on actual noisy near-term hardware. To this end, we deployed 10 toy instances of PO on the experimental 25-qubit trapped-ion quantum computer *Aria-1*³⁹. As an approximate solver, we executed a Trotterized adiabatic evolution to the Hamiltonians encoding QUBO reformulations (see Section “Experimental implementation” for implementation details). We executed a set of 10 random six-qubit instances with a fixed budget of 150 two-qubit gates for reformulations with M_{SDP} and M_{ℓ_1} . The limit on the circuit size and a suitably chosen maximal evolution time determine the number of Trotterization steps. The parameter choice ensures approximate adiabaticity. We find that the probability of measuring the optimal solution is more than an order of magnitude higher with the M_{SDP} reformulation than with M_{ℓ_1} [Fig. 2f]. The probability of measuring the optimal solution determines the required number of repetitions and, thus, enters inversely into the time-to-solution. This behavior is consistently observed across all instances. Figure 2g shows the average approximation ratio—quantifying the quality of an approximate solution—over all measured outcomes that satisfy the budget constraint per instance. The M_{SDP} formulations yield high ratios for most instances while the M_{ℓ_1} formulations perform comparable to classical uniform random sampling of solutions. Thus, already for small instances,

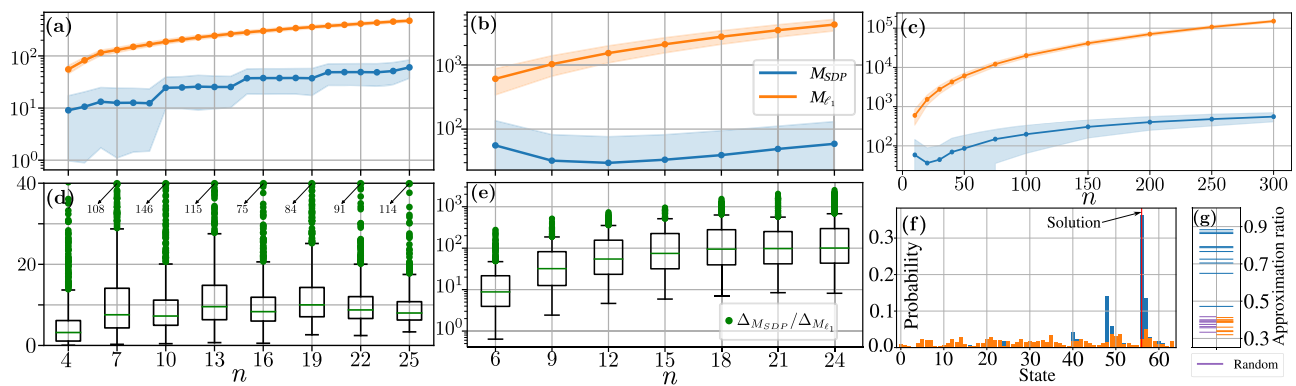


Fig. 2 | Numerical results. Panels **a** and **b** depict numerically calculated values of M_{SDP} (blue) and M_{L_1} (orange) for **a** sparse LCBOs (row-sparsity 5) and **b** portfolio optimization (PO, $w = 3$, with $N = n/w$ different stocks up to 8, $\gamma = 1$), averaged over 1000 instances for different problem sizes n . Shaded stripes indicate the standard deviation. We consistently find significantly smaller values for M_{SDP} . Panels **d** and **e** show box plots of the ratio $\Delta_{M_{SDP}}/\Delta_{M_{L_1}}$ of the spectral gaps for the instances in panels **a** and **b**, respectively. Green lines indicate the medians. The whiskers follow the 1.5 inter-quartile range convention (box contains half of the samples). The black arrows in panel **d** indicate the maximal achieved ratio that is beyond the scope of the displayed y-axis. The number at the arrows tip is the maximum value of these outliers. The spectral gaps of formulations with M_{SDP} are larger by factors of up to 100 for sparse LCBOs. For PO the factors increase with n , reaching 1000 for some instances. Panel **c** displays values for M_{SDP} (using a greedy heuristics) and M_{L_1} for

larger PO instances ($w = 5$, comprising up to $N = 60$ stocks, $\gamma = 1$) averaged over 100 instances. Shading indicates standard deviation. The gap between M_{SDP} and M_{L_1} grows with the system size. **f** Experimental results of 10 randomly-selected 6-qubit PO instances for a Trotterized adiabatic solver limited to 150 two-qubit gates on a trapped-ion IonQ quantum computer. Histogram of 1000 measurement outcomes of one of the 10 instances for the M_{SDP} and M_{L_1} reformulations. The optimal solution (red line) has probability larger than 0.3 for the M_{SDP} reformulation. In contrast, using M_{L_1} the experimental distribution is close to uniform. Panel **g** displays the experimentally obtained average approximation ratios for the 10 instances using both reformulations and when sampling uniformly at random from the feasible solutions. With the limited quantum resources of a state-of-the-art noisy quantum device, only instances using M_{SDP} outperform the random classical strategy.

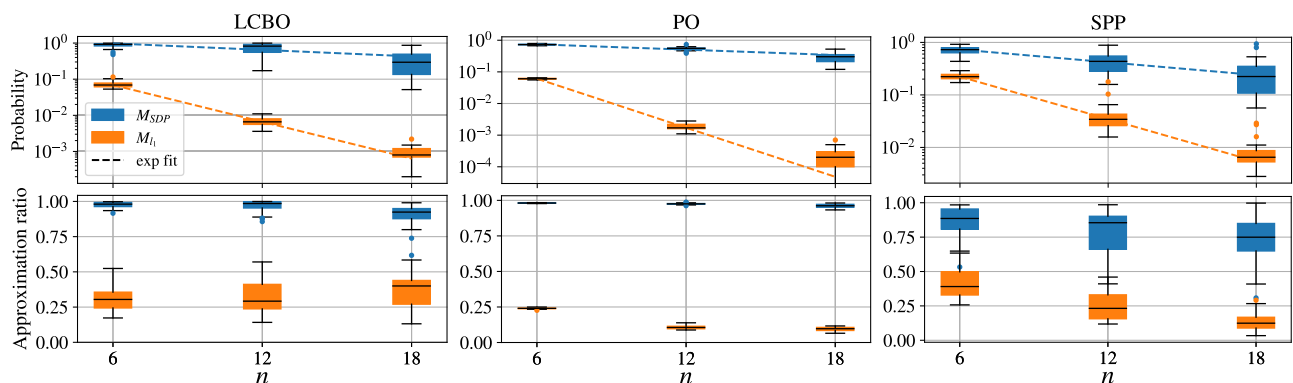


Fig. 3 | Simulation of Trotterized adiabatic evolution. The first and second rows show boxplots of, respectively, the probability of sampling the exact solution and the approximation ratio of the obtained state at the end of a simulation of a trotterized adiabatic-evolution (see Supplementary Information C for details) for different problem sizes n and QUBO reformulation using penalty weight M_{SDP} (blue) and M_{L_1} (orange). We use 25 random instances of one of the benchmark models (LCBO, PO, SPP) in each column. The whiskers follow the 1.5 inter-quartile range convention, the box contains half of the samples. In all settings, we observed that M_{SDP} reformulation leads to significantly high probability of obtaining the exact ground state and high approximation ratio, contrarily to M_{L_1} . In the first row, the dotted lines

represent exponential fits to the median of the probability with the form $p(n) = \alpha e^{-\beta n}$. The fitted decay factors β_{fit} are 0.07(4) and 0.390(4) for LCBO (blue and orange curves, respectively), 0.07(2) and 0.60(2) for PO, and 0.093(7) and 0.310(5) for SPP. The number in parentheses represents the standard deviation in the last digits of the corresponding value. We observe that also the ratio of the probabilities for M_{L_1} over M_{SDP} decreases exponentially with n , proportional to c^{-n} with c between 1.24 and 1.70 in our examples. Thus, we find a significantly increasing advantage in, e.g. sampling complexity, with the system size of the optimized penalization strategy over the baseline.

we find that using an optimized M is an essential prerequisite for deployment on noisy near-term hardware. Given the scaling observed in the numerical benchmarks, we expect that small values of M are even more important for the performances that are not dominated by noise on intermediate sizes hardware. Similar results have been observed in simulations of Trotterized adiabatic evolutions; see Fig. 3 and Supplementary Information C.

Discussion

On the conceptual side, we formalized the quantum big- M problem, giving rigorous definitions, establishing its computational complexity, and giving bounds on the impact of M on the spectral gap of a QUBO Hamiltonian.

The latter relates the big- M problem to performance guarantees of different solvers. From a practitioner's viewpoint, our main contribution is a versatile QUBO reformulation algorithm with enhanced spectral properties, based on the SDP relaxation. Our mindset is that classical solvers should be leveraged to pre-condition problems so as to exploit quantum hardware to its maximal potential—near-term devices in particular.

In numerical benchmarks, including Markovitz portfolio optimization (PO) instances from S&P 500 data, we consistently observe significant improvements in M and the spectral gap using the proposed algorithm. In a six-qubit proof-of-principle experiment with trapped ions, we find that these improvements translate into a tangible advantage in the probability of measuring the correct solution as well as the average solution quality. This

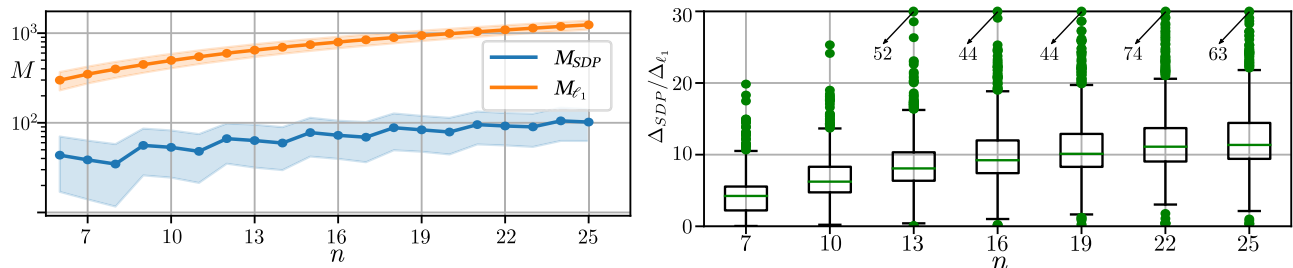


Fig. 4 | Set partitioning problem numerics. Big- M value (left) with two strategies, M_{F_1} and M_{SDP} , and ratio between spectral gaps resulting from the two choices (right), on a dataset of 1000 SPP instances generated with density = 0.25. The black

arrows in the panel on the right indicate the maximal ratio achieved by extreme outliers outside of the axis' scope.

being already present for small instances, the results presented in Fig. 1 support that the improvement due to a tighter reformulation will persist for larger instances.

Even when other big- M recipes are available, our method can assist such schemes, e.g. by providing a suitable starting point with tractable classical resources. For example, using multiple calls to the solver, one can determine a good value for M via binary search. After every call, M is increased when the returned solution is infeasible and otherwise decreased in smaller and smaller steps. Starting such a search from an M determined with our method still reduces the number of calls to a potentially expensive solver.

Beyond near-term quantum devices, our analysis of the Big- M problem also applies to future fault-tolerant quantum hardware, for instance in adiabatic schemes^{5,6,9} or quantum imaginary-time evolution simulations^{11–18}. Besides, a particularly interesting question to explore is how beneficial our general big- M recipe is for quantum-inspired, classical solvers^{22–24}. Combining our approach with modern scalable randomized algorithms for SDPs⁴⁰ can potentially further reduce the complexity of calculating optimized values for M . Finally, while our method was conceived for general instances, nuances of specific problems can enable heuristic tools for tighter lower bounds or feasible points, as already exemplified with the greedy heuristic for PO instances.

Methods

On the hardness of the quantum Big- M problem

Here we formally establish that determining an optimal M is in general as hard as finding the objective value of the original problem. We do this by proving a simple reduction from the decision-problem version of the latter to that of the former.

Finding the optimal objective value of a function f under constraints $\mathcal{C}$ is equivalent to an associated decision problem, decideF, which, given a threshold a and a gap $\delta > 0$, decides if $\min_{\mathbf{x} \in \mathcal{C}} f(\mathbf{x}) \leq a$ ('smaller') or if $\min_{\mathbf{x} \in \mathcal{C}} f(\mathbf{x}) \geq a + \delta$ ('greater'). Note that having access to an oracle for decideF allows one to efficiently find the optimal objective value via binary search. We want to relate the complexity of decideF to the following decision problem: *Given an instance of (P), an M , and a gap $\delta > 0$, decide if (P_M) with the given M is an exact reformulation of (P) with gap at least δ ('yes') or (P_M) fails to be an exact reformulation ('no').* We refer to this problem as decidePM. Note that decidePM is equivalent to the problem of finding the δ -optimal M : Given an optimal value for M , decidePM can be solved by comparing the M under scrutiny to the optimal one. In turn, with an oracle for decidePM, the optimal M can be found via binary search. Next, we prove the promised reduction.

Lemma 1. The problem decideF reduces to decidePM.

Proof. Consider an instance $(f, \{0, 1\}^n, a, \delta)$ of decideF. W.l.o.g., we assume that the instance is unconstrained. For the constraint problem there exist a polynomial reduction to an unconstrained problem, e.g. using (P_M) with the value of M defined in Eq. (2).

We will split decideF into decision problems where we decide the optimum for the subset with constant hamming weight $|\mathbf{x}| = \sum_{i=1}^n x_i = k \in \{0, 1, \dots, n\}$. Deciding for all $k \in \{0, 1, \dots, n\}$ individually if $\min_{|\mathbf{x}|=k} f(\mathbf{x}) \leq a$ ('smaller') or $\min_{|\mathbf{x}|=k} f(\mathbf{x}) \geq a + \delta$ ('greater') allows us to solve decideF in the following way. If all constant-Hamming weight decisions return 'greater', we also conclude 'greater' for decideF. If at least one constant-Hamming weight decision returns 'smaller', we return 'smaller' for decideF. It is straight-forward to see that this strategy solves decideF correctly in both cases.

It remains to reduce the decision problem with constant Hamming weight k to decidePM. For $k = 0$, i.e. $\mathbf{x} = \mathbf{0}$, we can directly solve the decision problem by evaluation. If we find $f(\mathbf{0}) \leq a$, we conclude 'smaller' for decideF. Thus, we can restrict our focus in the remainder to $k > 0$ and assume that $f(\mathbf{0}) > a$, where decideF is not yet decided. We choose $\alpha \geq \max_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}) - \min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}) + f(\mathbf{0}) - a$, e.g. $\alpha = M_{F_1} + f(\mathbf{0}) - a$ using M_{F_1} defined in Eq. (2) for the quadratic form Q defining f . Consider the following optimization problem:

$$\text{minimize}_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}) + \alpha |\mathbf{x}| |\mathbf{x}| - k \quad \text{subject to} \quad \mathbf{x} = \mathbf{0}. \quad (6)$$

The optimal point of (6) is the only feasible point $\mathbf{x}^* = \mathbf{0}$ with objective value $f(\mathbf{0})$. In other words, the constraint renders the optimization problem (6) trivial. Still of interest to us is the associated problem of deciding if certain values of M yield unconstrained reformulations of (6). As formulated (6) is not an instance of (P), since the objective function is not quadratic. But the optimization problem (6) can be recast as the following binary quadratic problem:

$$\text{minimize}_{\mathbf{x} \in \{0,1\}^n, p, m \in \{0, \dots, n\}} f(\mathbf{x}) + g(\mathbf{x}, p, m) \quad \text{subject to} \quad \mathbf{x} = \mathbf{0}. \quad (7)$$

with $g(\mathbf{x}, p, m) = \alpha |\mathbf{x}|(p + m) + \alpha(n^3 + 1)(p - m - |\mathbf{x}| + k)^2$. Here the non-negative integer variables p and m can each be encoded with $\lceil \log n \rceil$ binary variables. The last summand in g dominates the objective function for all values of $\mathbf{x}, p$ and m . Thus, at optimal p and m , it enforces the constraint $p - m = |\mathbf{x}| - k$. For $\mathbf{x} \neq \mathbf{0}$ the minimum of the objective function over p and m is attained when either p or m is equal to $|\mathbf{x}| - k$ while the other variable vanishes. We conclude that for all $\mathbf{x}$ the objective functions of (6) and (7) at optimal p and m coincide. Since (7) is an instance of (P), it defines instances of decidePM.

We now decide if (P_M) with $M = (f(\mathbf{0}) - a)/k > 0$ is an exact reformulation with gap δ of (7). If the answer of decidePM is 'yes' ('no'), we return 'greater' ('smaller'). Our claim is that this strategy correctly solves the decision problem for the minimum of f with constant Hamming-weight k .

To see this, let us first consider the case 'greater', where $\min_{|\mathbf{x}|=k} f(\mathbf{x}) \geq a + \delta$. By our choice of α , the QUBO reformulation of (7) with M

$= (f(\mathbf{0}) - a)/k > 0$ has an objective function that attains its minimum over the unfeasible points for $k = |\mathbf{x}|$. Thus, this minimum fulfills

$$\min_{\mathbf{x} \neq \mathbf{0}} \left\{ f(\mathbf{x}) + \alpha |\mathbf{x}| \|\mathbf{x}\| - k + \frac{f(\mathbf{0}) - a}{k} |\mathbf{x}| \right\} \\ = \min_{|\mathbf{x}|=k} f(\mathbf{x}) + f(\mathbf{0}) - a \geq \delta + f(\mathbf{0}). \quad (8)$$

Due to the trivializing constraint, $f(\mathbf{0})$ is the optimal value of (7). Hence, (8) establishes the criterion Eq. (1) for an exact reformulation. As required in this case, decidePM, thus, returns ‘yes’ and we decide correctly.

Second, let us consider the case ‘smaller’, i.e. $\min_{|\mathbf{x}|=k} f(\mathbf{x}) \leq a$. By the same argument as before, we now find that the minimum of the objective function of the QUBO reformulation over the unfeasible points is smaller or equal than $f(\mathbf{0})$. Thus, decidePM returns ‘no’ in this case. Using decidePM, we therefore always arrive at the correct decision about the minimum of f for constant Hamming-weight.

Since decideF encompasses NP-complete problems like 3SAT, as a corollary of Lemma 1, we establish that finding the optimal value of M is NP-hard.

Bounds on the spectral gap of Big- M QUBO reformulations

Next, we provide the detailed argument for *Observation 3* of the main text, and expand on some of its implications.

Let $H_M = H_f + MH_c$ be a Hamiltonian encoding of (P_M) . The normalized spectral gap of H_M is defined as

$$\Delta_M := \frac{E_1 - E_0}{E_{\max} - E_0}, \quad (9)$$

where E_0 , E_1 and $E_{\max}$ are the respective lowest, next-to-lowest and maximum energies of H_M . We will study the behavior of Δ_M compared to the corresponding quantity of the constraint optimization problem (P). To this end, let $\mathbf{x}^*$ be an optimal point as before and let further $\mathbf{x}_1^*$ be a next-to-optimal point of (P), i.e. the optimal point of (P) with the additional constraint $\mathbf{x} \notin f^{-1}(f(\mathbf{x}^*))$. Denote by $\mathcal{C} := \{\mathbf{x} | A\mathbf{x} = \mathbf{b}\}$ the constraint set and by $\bar{\mathcal{C}} := \{0, 1\}^n \setminus \mathcal{C}$ its complement. We define the two upper bounds of the shifted objective function $\bar{f} := \max_{\mathbf{x} \in \mathcal{C}} f(\mathbf{x}) - f(\mathbf{x}^*)$ and $\bar{f}_c := \max_{\mathbf{x} \in \bar{\mathcal{C}}} f(\mathbf{x}) - f(\mathbf{x}^*)$. We refer to

$$\Delta_0 := \frac{f(\mathbf{x}_1^*) - f(\mathbf{x}^*)}{\bar{f}} \quad (10)$$

as the *spectral gap* of (P).

(i) The Ising encoding ensures that $\langle \mathbf{x} | H_f | \mathbf{x} \rangle = f(\mathbf{x})$ for all $\mathbf{x}$, where $|\mathbf{x}\rangle$ denotes the basis vector that encodes the binary vector $\mathbf{x}$. For $\mathbf{x}$ feasible, $|\mathbf{x}\rangle$ is in the kernel of H_c . Thus, when M is chosen such that (P_M) is an exact reformulation of (P), we have $E_0 = f(\mathbf{x}^*)$.

(ii) Analogously, $f(\mathbf{x}_1^*)$ is still in the spectrum of H_M . Thus, $E_1 \leq f(\mathbf{x}_1^*)$. By Eq. (1) and assuming $\delta \leq \delta^* := f(\mathbf{x}_1^*) - f(\mathbf{x}^*)$, we have $f(\mathbf{x}^*) + \delta \leq E_1$. We conclude that $\delta \leq E_1 - E_0 \leq \delta^*$ (with the lower bound holding as long as δ does not exceed the upper bound). Note that the lower bound is saturated for the optimal M . Also $\bar{f} + f(\mathbf{x}^*)$ is still in the spectrum of H_M . Hence, $\bar{f} \leq E_{\max} - f(\mathbf{x}^*)$. All together, combined with (i) and its assumption, we arrive at the bound $\Delta_M = (E_1 - E_0)/(E_{\max} - E_0) \leq \delta^*/\bar{f} = \Delta_0$.

(iii) To infer the scaling of the spectral gap with M , we note that $E_{\max} - E_0 \geq \min\{f(\mathbf{x}) - f(\mathbf{x}^*)\} + M \|H_c\| \geq M \|H_c\| - M^*$, where we used the positivity of H_c on the infeasible subspace and denote by M^* the optimal M , i.e. the minimal M satisfying Eq. (1). Hence, $\Delta_0/\Delta_M \geq (E_{\max} - E_0)/\bar{f} \geq (M \|H_c\| - M^*)/\bar{f} \geq (M \|H_c\| - M^*)/\|H_f - E_0\|$. Thus, in particular $\Delta_M \in O(\Delta_0/M)$.

Benchmarked models

The present section illustrates the model definition and relevant details of the optimization problems tested, together with further results.

Random sparse LCBs. A general class of linearly-constrained binary quadratic optimization problems, whose formulation is (P), have been generated. We choose random instances for Q and A with a bounded row-sparsity s , i.e. $|\{j: Q_{ij} \neq 0\}| \leq s \forall i$, and similarly for A . The non-vanishing entries of Q , A and b are uniformly drawn at random. We let the number of constraints m grow linearly with the number of binary variables n , specifically, $m = \max\{\lfloor \frac{n}{5} \rfloor, 1\}$.

Set partitioning problem (SPP). Let $R_i \subset S$ be a subset of $S = \{1, \dots, m\}$ with an associated cost $c_i \geq 0$, for $i = 1, \dots, n$. A family of subsets $\{R_i\}_{i \in W}$ is a partition of S if $\bigcup_{i \in W} R_i = S$ and $R_i \cap R_j = \emptyset$ for all $i \neq j \in W$. The SPP consists of finding a partition of S with minimal total cost:

$$\text{minimize } \mathbf{c}^T \mathbf{x} \quad \text{subject to} \quad \sum_{i: \alpha \in R_i} x_i = 1 \quad \forall \alpha \in S. \quad (11)$$

The objective variables encode the subset family, with $x_i = 1$ if R_i is in the family and 0 otherwise. The constraints force the family to be a partition of S . We generate instances by randomly selecting constraint matrices A with fixed density, i.e. number of non-zero entries over total number of entries. Figure 3 shows simulations results relative to this problem class.

Portfolio Optimization (PO). The present paragraph illustrates how the data used in Portfolio Optimization instances were fetched from real data and adapted to Markowitz formulation (5). From stock market index S&P500, we downloaded the stock price history, referring to the 2 years period December 2020 until November 2022 with one-month interval, of 121 out of the 500 company stocks tracked by S&P500 (namely, the ones with no missing data in said intervals). Let us call $P_{t,a}$ such cost of an asset a , with time index t . The return at time step t is defined as

$$r_{t,a} = \frac{P_{t,a} - P_{t-1,a}}{P_{t-1,a}} \quad (12)$$

from which the expected return vector $\tilde{\boldsymbol{\mu}}$ and the covariance matrix $\tilde{\Sigma}$ can be computed as $\tilde{\boldsymbol{\mu}}_a = \frac{1}{T} \sum_{t=1}^T r_{t,a}$ and $\tilde{\Sigma}_{a,b} = \frac{1}{T-1} \sum_{t=1}^T (r_{t,a} - \tilde{\mu}_a)(r_{t,b} - \tilde{\mu}_b)$. We encode the real financial stock market data with decimal precision of 10^{-4} .

The number of stocks in an instance is determined by the partition number w^{37} , that describes the granularity of the portfolio discretization. The budget is divided in $2^w - 1$ equally large units. Each asset decision variable x_i is an integer that can take values from 0 up to $2^w - 1$, indicating how many of these partitions to allocate towards asset i . Therefore, we need w bits per asset and a instance of size n features $N = n/w$ different assets. The constraint $\sum_i x_i = 2^w - 1$ ensures that all $2^w - 1$ units are invested in one of the stocks. In the experiments we used a number of bits per asset $w \in \{2, 3, 5\}$. We generate random PO instances by sampling a subset of N assets from the 121 stocks with complete data uniformly at random.

Notice that $\tilde{\boldsymbol{\mu}}^T \mathbf{p}$ is the expected return of a portfolio if $\mathbf{p}$ represents the vector of the *portions* of the portfolio for each asset, i.e. $0 \leq p_i \leq 1$ and $\sum_i p_i = 1$. In order to have integer decision variables, the number of chunks $x_i = (2^w - 1)p_i$ is used, and in the final formulation (5) of the Markowitz model the factors are absorbed in the objective function, defining $\boldsymbol{\mu} = \tilde{\boldsymbol{\mu}}/(2^w - 1)$ and $\Sigma = \tilde{\Sigma}/(2^w - 1)^2$.

The last parameter that one needs to set to fully specify the instance is the risk aversion factor γ , weighting differently the return and the volatility in the objective function. In the experiments we used risk aversion factor $\gamma \in \{0.5, 1, 2\}$.

Experimental implementation

The adiabatic theorem⁵⁶ states that a quantum system will remain in its instantaneous ground state through small perturbations to its Hamiltonian. Adiabatic quantum computation exploits this fact by preparing a system under the Hamiltonian

$$H(s) = (1 - s)H_0 + sH_P, \quad (13)$$

where H_0 is a Hamiltonian with an easy to prepare ground state, H_P encodes the solution of a problem, and the schedule s is evolved from 0 to 1. If the evolution meets the conditions of the adiabatic theorem, the system will be at the ground state of H_P at the end of the evolution, hence solving the problem.

A QUBO instance can be mapped into an Ising Hamiltonian by promoting each binary variable x_i into quantum operators. Namely, by substituting x_i for $(1 - \sigma_i^z)/2$, where σ_i^j is the Pauli matrix j acting on qubit i . Our problem Hamiltonian reads $H_P = H_M + M H_C$, when combining the objective function $f(\mathbf{x})$ and the penalty term $M(A\mathbf{x} - \mathbf{b})^2$. This way, we recover the diagonal matrix of the QUBO instance. The initial Hamiltonian is usually chosen as $H_0 = -\sum_{i=1}^n \sigma_i^x$, as it has an easy to prepare ground state, the equal superposition of computational basis states. This is important, as one of the requirements for the evolution to work is a non-zero overlap between the initial and final ground states.

Deployment of algorithms on available quantum hardware requires precise fine-tuning as well as knowledge of the physical implementations of the device. In this work we target a gate-based ion trap quantum computer, as available through the IonQ cloud service³⁹. The native interactions available in the device are the following: Single qubit gates are fixed π and $\pi/2$ rotations along the X – Y plane, with precise control over the relative phase. Using this method, rotations around the Z axis are done virtually⁴¹, and incur no noise. The IonQ aria-1 device allows for partially-entangling Mølmer-Sørensen^{42,43} gates, that is, a precisely tuned two qubit rotation along the XX – YY plane with virtual control over the relative phases. Since the physics of the ion trap has access to a native XX interaction, we will perform a change of basis to the proposed Hamiltonian, so that the two-body terms in H_M are combinations of $\sigma_i^x \sigma_j^x$, and H_0 comprises σ_i^z terms. Crucially, the ground state of the initial Hamiltonian is the starting state of the device, resulting in an even easier preparation for the purposes of our evolution.

The adiabatic evolution, ideally performed by slowly sweeping over the interaction parameters of the device, will need to be Trotterized^{44,45}. By selecting a total evolution time and a discretization step, the evolution can be approximately reproduced by single and two-qubit gates acting on the quantum device. Moreover, this method can be used to control the amount of quantum resources dedicated to solving the problem, which provides an equal starting point to test different QUBO encodings.

In order to conform with the device specifications, and compare different QUBO reformulations under the same conditions, we limit the number of two-qubit gates to 150. This corresponds to a final annealing time of 100 with a Trotterization step of 10 for the instances considered. The parameters used are far from an ideal adiabatic evolution, however, they should still result in an amplification of the ground state of the problem. As shown in Fig. 2f, this amplification is only significant using the M_{SDP} reformulation, making it indispensable even for current noisy devices. This can also hint at advantages on more complex algorithms such as QAOA or VQE when encoding the problem using the proposed M_{SDP} reformulation.

In approximate optimization, outputs that reach a high value for the objective function are desirable even if they do not maximize it. In order to quantify the quality of the solutions, we will use an approximation ratio. We define the approximation ratio as $\alpha(\mathbf{x}) = (f(\mathbf{x}) - f(\mathbf{x}_{max})) / (f(\mathbf{x}^*) - f(\mathbf{x}_{max}))$ if $\mathbf{x}$ satisfies the constraints.

We build the Hamiltonian for the presented problem instances and Trotterize them using the quantum simulation library Qibo^{46,47}. Then, the resulting quantum circuits are parsed into native gate instructions for the IonQ aria-1 device. Code to reproduce this procedure is made available in the following Github repository⁴⁸. To confirm that this behavior remains consistent as the instances grow, we further simulate exact Trotterized adiabatic evolutions on instances with 12 and 18 qubits, see Fig. 4 and Supplementary Information C. For all models and problem sizes, we consistently observe significantly improved probabilities for the optimal solution and larger approximation ratios using M_{SDP} over the baseline with M_{ℓ_1} .

Greedy algorithm for portfolio optimization

Any strategy to get a feasible point $\mathbf{x}_{feas}$ using classical resources is a viable option to obtain M via Eq. (3). For various classes of optimization problems, it is possible to apply a greedy heuristic algorithm to efficiently obtain a quasi-optimal point. To exemplify this, we describe a straight-forward greedy strategy for instances of Portfolio Optimization (5). Recall, that given N assets and a partition number w , the portfolio is discretized into $2^w - 1$ equal fractions. The following algorithm aims at obtaining solutions by systematically allocating each portfolio portion to the asset that minimizes the objective function when evaluated on the existing segment of the portfolio.

Algorithm 1. GreedyPortfolio ($\Sigma, \mu, \gamma, N, w$)

```

input : Risk matrix  $\Sigma$ , expected return  $\mu$ , risk aversion
        factor  $\gamma$ .
        Number of assets  $N$ .
        Partition number  $w$ .
1   $\mathbf{x} \leftarrow \mathbf{0} \in \mathbb{Z}^N$  // Initialize empty portfolio
2  for  $i \leftarrow 1 \dots 2^w - 1$  do
3    for  $k \leftarrow 1 \dots N$  do
4       $\tilde{\mathbf{x}} \leftarrow \mathbf{x}$ 
5       $\tilde{x}_k \leftarrow \tilde{x}_k + 1$ 
6       $f' \leftarrow -\mu^t \tilde{\mathbf{x}} + \gamma \tilde{\mathbf{x}}^t \Sigma \tilde{\mathbf{x}}$ 
7      if  $k = 1$  or  $f' < f^*$  then
8         $k^* \leftarrow k$ 
9         $f^* \leftarrow f'$ 
10    $x_{k^*} \leftarrow x_{k^*} + 1$  // Assign a unit to best
        asset
11 return optimized portfolio  $\mathbf{x}$ 

```

Data availability

The datasets generated and analyzed for the IonQ experiment implementation are available in the quantum-bigm-trotterization repository <https://github.com/igres26/quantum-bigM-trotterization/tree/main/data>⁴⁸. All the other datasets generated and analysed during the current study are available in the qubo_mapper repository, https://github.com/EdoardoAlessandroni/qubo_mapper/tree/master/problems⁴⁹.

Received: 13 June 2024; Accepted: 20 June 2025;

Published online: 26 July 2025

References

1. Abbas, A. et al. Quantum optimization: potential, challenges, and the path forward. Preprint at <https://arxiv.org/abs/2312.02279> (2023).
2. Montanaro, A. Quantum algorithms: an overview. *NPJ Quantum Inf.* **2**, 15023 (2015).
3. Durr, C. & Hoyer, P. A quantum algorithm for finding the minimum. Preprint at <https://arxiv.org/abs/quant-ph/9607014> (1996).
4. Ambainis, A. et al. Quantum speedups for exponential-time dynamic programming algorithms. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1783–1793 (SIAM, 2019).
5. Farhi, E., Goldstone, J., Gutmann, S., and Sipser, M. Quantum computation by adiabatic evolution. Preprint at <https://arxiv.org/abs/quant-ph/0001106> (2000).
6. Albash, T. & Lidar, D. A. Adiabatic quantum computation. *Rev. Mod. Phys.* **90**, 015002 (2018).
7. Lang, J., Zielinski, S. & Feld, S. Strategic portfolio optimization using simulated, digital, and quantum annealing. *Appl. Sci.* **12**, 12288 (2022).
8. Salatino, G., Matzler, M., Scocco, A., Lucignano, P., & Passarelli, G. Noise effects on diabatic quantum annealing protocols. Preprint at <https://arxiv.org/abs/2502.07588> (2025).

9. Nagies, S. et al. Boosting quantum annealing performance through direct polynomial unconstrained binary optimization. *Quantum Sci. Technol.* **10**, 035008 (2025).
10. Hegde, P. R., Passarelli, G., Scocco, A. & Lucignano, P. Genetic optimization of quantum annealing. *Phys. Rev. A* <https://doi.org/10.1103/physreva.105.012612> (2022).
11. McArdle, S. et al. Variational ansatz-based quantum simulation of imaginary time evolution. *NPJ Quantum Inf.* **5**, 75 (2019).
12. Motta, M. et al. Determining eigenstates and thermal states on a quantum computer using quantum imaginary time evolution. *Nat. Phys.* **16**, 205 (2020).
13. Nishi, H., Kosugi, T. & Matsushita, Y. Implementation of quantum imaginary-time evolution method on nisc devices by introducing nonlocal approximation. *NPJ Quantum Inf.* **7**, 85 (2021).
14. Poulin, D. & Wocjan, P. Sampling from the Thermal Quantum Gibbs State and Evaluating Partition Functions with a Quantum Computer. *Phys. Rev. Lett.* **103**, 220502 (2009).
15. Chowdhury, A. N. & Somma, R. D. Quantum algorithms for gibbs sampling and hitting-time estimation. *Quantum Inf. Comput.* **17**, 41 (2017).
16. Wang, Y., Li, G. & Wang, X. Variational quantum gibbs state preparation with a truncated taylor series. *Phys. Rev. Appl.* **16**, 054035 (2021).
17. Silva, T. d. L., Taddei, M. M., Carrazza, S. & Aolita, L. Fragmented imaginary-time evolution for early-stage quantum signal processors. *Sci. Rep.* **13**, 18258 (2023).
18. Kyaw, T. H. et al. Boosting quantum amplitude exponentially in variational quantum algorithms. *Quantum Sci. Technol.* **9**, 01LT01 (2023).
19. Farhi, E., Goldstone, J., & Gutmann, S. A quantum approximate optimization algorithm. Preprint at <https://arxiv.org/abs/1411.4028> (2014).
20. Basso, J., Farhi, E., Marwaha, K., Villalonga, B., & Zhou, L. The quantum approximate optimization algorithm at high depth for MaxCut on large-girth regular graphs and the Sherrington-Kirkpatrick model. In *17th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2022)*, vol. 232, 7:1–7:21, <https://doi.org/10.4230/LIPIcs.TQC.2022.7> (2022).
21. He, Z. et al. Alignment between initial state and mixer improves QAOA performance for constrained optimization. *npj Quantum Inf.* **9**, 121 (2023).
22. Goto, H., Tatsumura, K. & Dixon, A. R. Combinatorial optimization by simulating adiabatic bifurcations in nonlinear hamiltonian systems. *Sci. Adv.* **5**, eaav2372 (2019).
23. Kanao, T. & Goto, H. Simulated bifurcation assisted by thermal fluctuation. *Nat. Commun. Phys.* **5**, 153 (2022).
24. Mohseni, N., McMahon, P. L. & Byrnes, T. Ising machines as hardware solvers of combinatorial optimization problems. *Nat. Rev. Phys.* **4**, 363–379 (2022).
25. Qiskit documentation, Converters for quadratic programs - linearequalitytopenalty. https://qiskit.org/ecosystem/optimization/tutorials/02_converters_for_quadratic_programs.html#LinearEqualityToPenalty (2022).
26. Iosue, J. T., Welcome to qubover's documentation! <https://qubover.readthedocs.io/en/latest/> (2020).
27. Zaman, M., Tanahashi, K. & Tanaka, S. Pyqubo: Python library for mapping combinatorial optimization problems to qubo form. *IEEE Trans. Comput.* **71**, 838 (2021).
28. Karimi, S. & Ronagh, P. Practical integer-to-binary mapping for quantum annealers. *Quantum Inf. Process.* **18**, 1 (2019).
29. Harwood, S. et al. Formulating and solving routing problems on quantum computers. *IEEE Trans. Quantum Eng.* **2**, 1 (2021).
30. Azad, U., Behera, B. K., Ahmed, E. A., Panigrahi, P. K. & Farouk, A. Solving vehicle routing problem using quantum approximate optimization algorithm. *IEEE Trans. Intell. Transp. Syst.* **24**, 7564 (2023).
31. Leonidas, I. D., Dukakis, A., Tan, B. & Angelakis, D. G. Qubit efficient quantum algorithms for the vehicle routing problem on noisy intermediate-scale quantum processors. *Adv. Quantum Technol.* **7**, 2300309 (2024).
32. Goemans, M. X. & Williamson, D. P. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. ACM* **42**, 1115 (1995).
33. Ebadi, S. et al. Quantum optimization of maximum independent set using rydberg atom arrays. *Science* **376**, 1209 (2022).
34. Lucas, A. Ising formulations of many np problems. *Front. Phys.* **2**, 5 (2014).
35. Ayodele, M., Penalty weights in qubo formulations: permutation problems. In *Evolutionary computation in combinatorial optimization*, vol. 13222, 159–174 (Springer International Publishing, 2022).
36. Markowitz, H. Portfolio selection. *J. Financ.* **7**, 77 (1952).
37. Grant, E., Humble, T. S. & Stump, B. Benchmarking quantum annealing controls with portfolio optimization. *Phys. Rev. Appl.* **15**, 014012 (2021).
38. Rosenberg, G. et al. Solving the optimal trading trajectory problem using a quantum annealer. *IEEE J. Sel. Top. Signal Process.* **10**, 1053 (2016).
39. Ionq trapped ion quantum computing. <https://ionq.com/>. (2023)
40. Yurtsever, A., Tropp, J. A., Fercoq, O., Udell, M. & Cevher, V. Scalable semidefinite programming. *SIAM J. Math. Data Sci.* **3**, 171 (2021).
41. McKay, D. C., Wood, C. J., Sheldon, S., Chow, J. M. & Gambetta, J. M. Efficient z gates for quantum computing. *Phys. Rev. A* **96**, 022330 (2017).
42. Mølmer, K. & Sørensen, A. Multiparticle entanglement of hot trapped ions. *Phys. Rev. Lett.* **82**, 1835 (1999).
43. Solano, E., de Matos Filho, R. L. & Zagury, N. Deterministic bell states and measurement of the motional state of two trapped ions. *Phys. Rev. A* **59**, R2539 (1999).
44. Trotter, H. F. On the product of semi-groups of operators. *Proc. Am. Math. Soc.* **10**, 545 (1959).
45. Hatano, N. & Suzuki, M. Finding exponential product formulas of higher orders. In *Quantum annealing and other optimization methods*, vol. 679, 37–68 (Springer Berlin Heidelberg, 2005).
46. Efthymiou, S. et al. Qibo: a framework for quantum simulation with hardware acceleration. *Quantum Sci. Technol.* **7**, 015018 (2021).
47. The Qibo team. qiboteam/qibo: Qibo 0.1.15 <https://doi.org/10.5281/zenodo.8093455> (2023)
48. Ramos-Calderer, S. quantum-bigm-trotterization <https://github.com/igres26/quantum-bigm-trotterization> (2023).
49. Alessandrini, E. qubo_mapper https://github.com/EdoardoAlessandrini/qubo_mapper (2023).

Author contributions

E.A. implemented the numerics. S.R. conducted the experimental deployment on IonQ. E.A. and S.R. derived the analytical scaling of the gap. I.R. proved the NP-hardness of finding the optimal big-M. I.R., E.T., and L.A. conceived the project and provided guidance in all steps. All authors contributed to the conception and write-up of the paper.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41534-025-01067-0>.

Correspondence and requests for materials should be addressed to Edoardo Alessandrini.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2025